Group ni Alex

Lunti: Smart Personal Finance Management
Software Design Document
Version 01.00

Revision History

Date	Version	Description	Author
09/05/2026	01.00	Initial SDD based on the Lunti_SRS 01.00	Atayde, Ygot, Birondo, Enriquez

Table of Contents

1. INTRODUCTION 4
1.1 Purpose 4
1.2 Scope 4
1.3 Overview 4
1.4 Reference Material 4
1.5 Definitions and Acronyms 5

2. SYSTEM OVERVIEW 5

3. SYSTEM ARCHITECTURE 6
3.1 Decomposition Description 7
3.2 Design Rationale 8

4. DATA DESIGN 8
4.1 Data Description 9
4.2 Data Dictionary 9

5. COMPONENT DESIGN 11

6. HUMAN INTERFACE DESIGN 14
6.1 Overview of User Interface 14
6.2 Screen Images 14
6.3 Screen Objects and Actions 15

7. REQUIREMENTS MATRIX 15

1. INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) describes the software architecture, subsystem decomposition, data design, component design, and interface design of Lunti, a smart

personal finance and grocery management mobile application. It translates the functional and non-functional requirements defined in the Lunti Software Requirements Specification (SRS, v01.00) into a concrete technical design that development, QA, and future maintenance teams can implement and verify against.

The intended audience for this document includes software developers, QA engineers, UI/UX designers, project stakeholders, and evaluators who need a shared technical reference for how Lunti is built.

1.2 Scope

The design covers the mobile-first Lunti application and its supporting backend: expense recording, receipt scanning and OCR extraction, budget management, spending analytics, purchase pattern tracking, smart replenishment, smart shopping lists, ingredient management, recipe recommendations, AI-powered insights, and user feedback/personalization.

The design connects a React Native mobile client to a Python/FastAPI backend, a PostgreSQL database (hosted on Supabase), and external services for authentication, file storage, OCR, AI inference, and push notifications. Advanced bank integrations, multi-currency support, shared/family accounts, and nutrition tracking are explicitly out of scope for this design, consistent with the SRS.

1.3 Overview

Section 2 gives a general description of the system. Section 3 presents the system architecture, including the architectural diagram, subsystem decomposition, and design rationale. Section 4 describes the data design, including the data model and data dictionary. Section 5 describes the internal design of key components using pseudocode. Section 6 covers the human interface design. Section 7 provides a requirements traceability matrix back to the SRS. Section 8 contains supporting appendices.

1.4 Reference Material

Lunti: Smart Personal Finance Management — Software Requirements Specification, v01.00.

1.5 Definitions and Acronyms

Term	Definition
API	Application Programming Interface.

OCR	Optical Character Recognition, used to extract text from receipt images.
RBAC	Role-Based Access Control.
JWT	JSON Web Token, used to represent an authenticated session.
ORM	Object-Relational Mapping (SQLAlchemy, used by the FastAPI backend).
MVP	Minimum Viable Product.
ERD	Entity-Relationship Diagram.
DTO	Data Transfer Object — the shape of data exchanged between client and API.

2. SYSTEM OVERVIEW

Lunti is a mobile-based personal finance and grocery management application that links financial tracking to everyday purchasing and food-related behavior. Rather than only recording how much a user spends, Lunti analyzes recorded transactions to understand what the user buys, how often, when a product is likely to need replenishment, and what meals can be prepared from ingredients the user already has.

The system follows the pipeline: Scan/Input → Track → Analyze → Learn → Predict → Remind → Plan → Cook → Save, connecting Money → Purchases → Groceries → Ingredients → Meals. The mobile client (React Native + TypeScript) is the sole interface for regular users; a Python/FastAPI backend implements all business logic, deterministic financial calculations, and orchestration of external AI/OCR services, backed by a PostgreSQL database hosted on Supabase.

A key design principle, driven directly by SRS constraints 3.6.3 and 3.6.4, is the separation of deterministic logic from probabilistic/AI logic: budget totals, remaining balances, and other financial calculations are always computed by application code against stored transaction data, never inferred by the AI model. AI (OpenAI API) and OCR (Google Cloud Vision) are used only for extraction and interpretation tasks, and their output is always surfaced to the user for confirmation before it affects stored data.

3. SYSTEM ARCHITECTURE

Lunti uses a layered client-server architecture: a mobile presentation layer, a modular backend service layer exposing a single REST API, a relational data layer, and a set of

external service integrations. The backend is implemented as a modular monolith (a single FastAPI deployment organized into cohesive service modules) rather than separate microservices, which keeps operational overhead low for the MVP while preserving clear internal module boundaries that could be extracted into independent services later if scale requires it.

The major subsystems are:

- Mobile Application (Presentation Layer) — React Native + TypeScript app handling UI, local state (Zustand), server-state caching (TanStack Query), and the device camera interface.
- API Layer — FastAPI entry point handling routing, request validation, authentication/session verification, and role-based authorization (RBAC).
- Expense & Budget Service — transaction CRUD, budget CRUD, deterministic budget/remaining-balance calculations, and budget-threshold notifications.
- Receipt/OCR Orchestration Service — manages the receipt image lifecycle: upload, OCR invocation, parsing raw OCR output into structured draft transactions, and handoff to the client for confirmation.
- Analytics & Pattern Service — computes spending summaries, category breakdowns, trends, and purchase-pattern statistics (frequency, interval, recurring products) from transaction history.
- Replenishment & Shopping List Service — consumes purchase-pattern output to generate replenishment suggestions and to assemble/enhance the smart shopping list from purchase history, inventory, meal plans, and budget.
- Ingredient & Recipe Service — maintains the estimated ingredient inventory and matches available ingredients against a recipe catalog to produce recommendations.
- AI Insights Service — sends aggregated, de-identified-where-possible summaries of spending/purchasing data to the OpenAI API and returns interpreted, human-readable insights.
- Notification Service — generates and dispatches budget, replenishment, shopping, and meal-recommendation notifications through Firebase Cloud Messaging.
- Data Layer — PostgreSQL database (Supabase-hosted) storing all persistent application data.
- External Services — Supabase Auth (authentication/session), Supabase Storage (receipt images), Google Cloud Vision API (OCR), OpenAI API (AI insights and recipe reasoning), Firebase Cloud Messaging (push delivery).

All client-to-backend traffic goes through the API Layer over HTTPS/JSON. The API Layer verifies the Supabase-issued session token on every request before delegating to a domain service. Each domain service is the only path to its portion of the PostgreSQL schema, which

keeps ownership of data access rules unambiguous.

3.1 Decomposition Description

This is a functional decomposition: each backend service listed in 3.1 maps directly to one or more SRS functional areas (SRS 3.1.1–3.1.11). The table below shows this mapping at a glance; Section 7 provides the full requirements traceability.

Subsystem	SRS Functional Area(s) Covered
Expense & Budget Service	3.1.1 Expense Management, 3.1.3 Budget Management
Receipt/OCR Orchestration Service	3.1.2 Receipt Scanning and OCR
Analytics & Pattern Service	3.1.4 Spending Analytics, 3.1.5 Purchase Pattern Tracking
Replenishment & Shopping List Service	3.1.6 Smart Replenishment, 3.1.7 Smart Shopping List
Ingredient & Recipe Service	3.1.8 Ingredient Management, 3.1.9 Recipe Recommendations
AI Insights Service	3.1.10 AI-Powered Insights, 3.1.11 User Feedback and Personalization (feedback consumption)
Notification Service	Cross-cutting: budget alerts, replenishment/shopping reminders, meal recommendations (SRS §3.7.5)

As a representative cross-subsystem flow, the diagram below traces the receipt-scanning use case (SRS 3.1.2) end to end, since it involves the mobile client, two backend services, an external OCR provider, cloud storage, and the database.

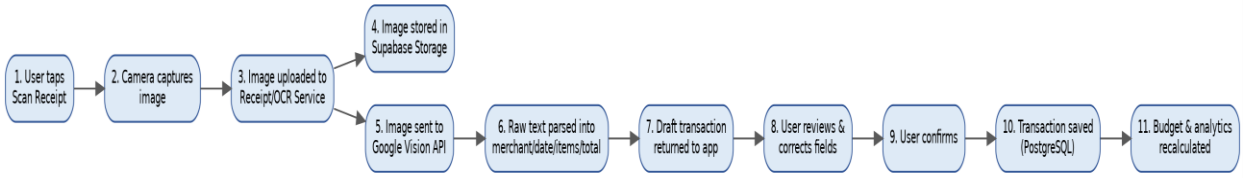


Figure 3.2 — Receipt scan → OCR → confirmed transaction flow.

Steps 8–9 are mandatory per SRS assumption 17 and requirement 3.1.2: extracted data is always shown to the user for verification/correction and is only persisted after explicit confirmation, ensuring OCR error never silently corrupts financial records.

3.2 Design Rationale

Modular monolith vs. microservices: A microservice-per-domain approach was considered, since the SRS decomposes cleanly into services (expense, OCR, analytics, replenishment, ingredients, insights). It was rejected for the MVP because the team is small, request volume is modest (single-tenant personal finance app), and cross-service transactions (e.g., a confirmed receipt must atomically update transactions, budgets, and pattern statistics) are simpler inside one process with one database transaction. The chosen modular monolith keeps each domain in its own package with a defined internal interface, so extraction into standalone services remains possible later without a rewrite.

Deterministic engine separate from AI: Budget totals and remaining balances are computed by plain application logic against PostgreSQL data rather than by asking the AI model to "interpret" a balance. This was a direct response to SRS constraint 3.6.4 (AI limitations) — it guarantees financial figures are reproducible and auditable, while AI is confined to advisory, non-authoritative outputs (insights, recipe suggestions, replenishment confidence).

Managed services over self-hosted equivalents: Supabase (Auth, Storage, Postgres hosting) and Firebase Cloud Messaging were chosen over self-managed alternatives to reduce operational burden for a small team building an MVP, at the cost of some vendor lock-in. Google Cloud Vision and OpenAI were chosen over building in-house OCR/ML models because the SRS explicitly scopes AI/OCR as external-service capabilities (2.5) and because in-house models would require far more training data and MLOps investment than an MVP justifies.

Confirmation-gated writes: Both the receipt pipeline and AI-driven suggestions (replenishment, shopping list) write nothing to authoritative tables until the user confirms or acts on them. This trades some automation for the data-reliability guarantee required by SRS 3.3.1 and the OCR/AI limitation constraints in 3.6.3–3.6.4.

4. DATA DESIGN

4.1 Data Description

Lunti's information domain is transformed into a normalized relational schema in PostgreSQL. Each SRS entity (Transaction, Budget, Product, Purchase Pattern, Ingredient Inventory, Smart Shopping List, Recipe) maps to one or more tables. Receipt images themselves are not stored in PostgreSQL; only a reference URL to the object in Supabase Storage is stored, keeping the relational database limited to structured data. The core entities and their relationships are:

- A User has many Transactions, Budgets, Shopping Lists, Ingredient Inventory

records, and Feedback entries.

- A Transaction belongs to a Category and may originate from a Receipt; it contains one or more Transaction Line Items (individual products/prices from a receipt).
- A Product is identified across Transaction Line Items to build Purchase Pattern statistics per user, which feed Replenishment Suggestions.
- A Shopping List has many Shopping List Items, which may reference a Product and/or a Recipe Ingredient requirement.
- A Recipe has many Recipe Ingredients, which are matched against a user's Ingredient Inventory to compute available/missing ingredients.
- An Insight and a Feedback entry both reference a User and, where applicable, the Suggestion or Insight they relate to, closing the personalization feedback loop (SRS 3.1.11).

All monetary values are stored as fixed-point decimals (not floating point) to avoid rounding errors in budget arithmetic. All tables include created_at/updated_at timestamps to support auditability and the data-reliability requirement in SRS 3.3.1.

4.2 Data Dictionary

Entity	Key Attributes	Description
User	id (PK), email, password_hash, role, created_at	A registered account; role distinguishes Regular User from Administrator (SRS §2.3).
Category	id (PK), name, is_default	Expense classification used for manual entry and automatic categorization.
Transaction	id (PK), user_id (FK), category_id (FK), amount, date, merchant, description, source (manual/receipt), receipt_id (FK, nullable)	A recorded financial activity (SRS §3.1.1).
TransactionLineItem	id (PK), transaction_id (FK), product_id (FK, nullable), name, quantity, unit_price, discount, tax	Individual product line extracted from a receipt or entered manually.
Receipt	id (PK), user_id (FK), image_url, ocr_raw_text, ocr_status, confirmed_at	Metadata for a scanned receipt image and its OCR result (SRS §3.1.2).

Budget	id (PK), user_id (FK), category_id (FK, nullable), period_type (daily/weekly/monthly/custom), start_date, end_date, limit_amount	A spending limit for a period and optional category (SRS §3.1.3).
Product	id (PK), user_id (FK), name, is_food, default_category_id (FK)	A distinct purchasable item tracked over time for pattern analysis (SRS §3.1.5).
PurchasePattern	id (PK), user_id (FK), product_id (FK), avg_interval_days, last_purchased_at, purchase_count, confidence_score	Derived statistics describing a recurring purchase behavior (SRS §3.1.5).
ReplenishmentSuggestion	id (PK), user_id (FK), product_id (FK), predicted_date, status (pending/added/dismissed/modified)	A generated prediction that a product may need repurchasing (SRS §3.1.6).
ShoppingList	id (PK), user_id (FK), name, created_at	A user's shopping list container (SRS §3.1.7).
ShoppingListItem	id (PK), shopping_list_id (FK), product_id (FK, nullable), recipe_id (FK, nullable), name, quantity, is_purchased, source (manual/suggested)	An individual item on a shopping list.
IngredientInventory	id (PK), user_id (FK), product_id (FK), estimated_quantity, estimated_expiry, confirmed_by_user	Estimated food ingredients currently available to the user (SRS §3.1.8).
Recipe	id (PK), name, servings, prep_time_minutes	A recipe available for recommendation (SRS §3.1.9).
RecipeIngredient	id (PK), recipe_id (FK), product_id (FK, nullable), name, quantity_required	A single ingredient requirement of a recipe.
Insight	id (PK), user_id (FK), type, message, generated_at, dismissed	An AI-generated, data-grounded observation about the user's finances (SRS §3.1.10).
Feedback	id (PK), user_id (FK), target_type, target_id, action (add_to_list/still_have/bought_elsewhere/dont_remind/dismiss)	User response to a suggestion/insight, used to refine future predictions

		(SRS §3.1.11).
Notification	id (PK), user_id (FK), type, payload, sent_at, read_at	A dispatched push notification record (SRS §3.7.5).

5. COMPONENT DESIGN

This section describes the internal processing logic of the major backend components identified in Section 3.2. The descriptions focus on the functions that are specific to LUNTI and explain how each component processes data to provide the required system functionality. Standard database operations, such as basic create, read, update, and delete operations, are not described in detail.

5.1 Receipt OCR Processing (Receipt/OCR Orchestration Service)

The Receipt/OCR Orchestration Service is responsible for processing receipt images submitted by users and converting them into structured purchase information. When a user uploads or captures a receipt, the system first stores the image in the configured cloud storage and creates a receipt record associated with the user's account. The receipt is initially assigned a processing status while the OCR operation is being performed.

The service then sends the receipt image to the configured Optical Character Recognition (OCR) service, Google Cloud Vision, to extract the text contained in the receipt. The extracted information is evaluated to determine whether the result is usable. If the OCR operation fails or produces information with insufficient confidence, the receipt is marked as failed and the user is prompted to retry the scan or manually enter the information.

If the OCR result is acceptable, the system parses the extracted text to identify relevant receipt information, including the merchant name, transaction date, individual line items, taxes, and total amount. The resulting information is returned to the mobile application as a draft. At this stage, no financial transaction is created because the user must first review and confirm the extracted information.

When the user confirms the receipt, the system validates the corrected information to ensure that required fields are present, monetary values are valid, and the transaction date is correctly formatted. The confirmed information is then processed as a single database transaction. The system creates the corresponding transaction and transaction line items, changes the receipt status to confirmed, recalculates the user's applicable budgets, and updates the purchase-pattern information based on the items contained in the receipt.

5.2 Deterministic Budget Calculation (Expense and Budget Service)

The Expense and Budget Service is responsible for calculating the user's budget status based on recorded transactions. The calculation is deterministic and does not depend on the AI service.

When a transaction is recorded or confirmed, the system identifies all active budgets that apply to the transaction date. For each applicable budget, the system calculates the total amount spent during the budget period. If a budget is

associated with a specific category, only transactions belonging to that category are included in the calculation.

The system then subtracts the calculated spending from the budget limit to determine the remaining amount. If the remaining amount is zero or negative, the system identifies the budget as exceeded and generates a budget-exceeded notification. If the budget has not been exceeded but spending has reached at least ninety percent of the configured limit, the system generates a near-limit notification.

All calculations are performed using stored transaction data and database aggregation. This ensures that financial totals and budget status are predictable, consistent, and independent of AI-generated results.

5.3 Purchase Pattern Detection (Analytics and Pattern Service)

The Analytics and Pattern Service identifies recurring purchasing behavior from the user's historical purchase records. When new purchase information becomes available, the system evaluates each purchased item and associates it with an existing product record or creates a new product record when necessary.

The service retrieves the user's historical purchases for each product and evaluates whether enough historical information exists to identify a meaningful purchasing pattern. When the number of previous purchases is below the minimum required sample size, the system marks the pattern as having insufficient data and does not attempt to make a reliable prediction.

When sufficient purchase history is available, the system calculates the intervals between consecutive purchases. It then determines the average number of days between purchases and measures the variability of those intervals. The confidence of the detected pattern is based on the amount of available historical data and the consistency of the purchasing intervals. More purchase samples and more consistent intervals result in greater confidence.

The resulting purchase pattern is stored or updated with information such as the associated product, estimated average purchase interval, confidence level, and most recent purchase date. These values are subsequently used by the Smart Replenishment component.

5.4 Smart Replenishment Prediction (Replenishment and Shopping List Service)

The Replenishment and Shopping List Service uses detected purchase patterns to identify items that may need to be purchased again. The system evaluates the purchase patterns associated with the user's account and ignores patterns whose confidence level is below the configured threshold. This prevents the application from generating unnecessary reminders for irregular or insufficiently supported purchases.

For patterns that meet the required confidence level, the system calculates an expected next purchase date by adding the estimated average purchase interval to the date of the user's most recent purchase. The system then compares the expected date with the current date.

When the expected purchase date falls within the configured approach window, the system creates or updates a replenishment suggestion. The suggestion contains information such as the item, predicted purchase date, and pending status. The user can then add, dismiss, or modify the suggestion through the mobile application.

User actions on replenishment suggestions are recorded as feedback and are used as additional information for future personalization and confidence evaluation. This allows the replenishment functionality to become more relevant as the system collects more information about the user's purchasing behavior.

5.5 Recipe Matching (Ingredient and Recipe Service)

The Ingredient and Recipe Service generates recipe recommendations based on the ingredients currently available to the user. The system first retrieves the user's ingredient inventory and compares it with the ingredient requirements of recipes contained in the recipe catalog.

For each recipe, the system identifies which required ingredients are already available in the user's inventory and which ingredients are missing. A match score is then calculated based on the proportion of required ingredients that are already available. Recipes with a higher proportion of available ingredients receive higher match scores.

The system also estimates the potential cost of preparing each recipe by calculating the typical price of the ingredients that are missing from the user's inventory. When applicable, the estimated cost is compared with the user's remaining budget.

The resulting recipes are ranked according to their ingredient match score and filtered according to applicable budget constraints. The highest-ranking recipes are then presented to the user as personalized recommendations. Missing ingredients can subsequently be added to the user's shopping list.

5.6 AI-Powered Insight Generation (AI Insights Service)

The AI Insights Service provides personalized explanations and recommendations based on information already recorded in LUNTI. Before requesting an AI-generated insight, the system first calculates a structured summary of the user's available data. This summary may include spending by category, purchase patterns, and current budget status.

The system first checks whether sufficient information is available to generate a meaningful insight. If the user's data is insufficient, the system returns an appropriate placeholder indicating that more information is required rather than generating an unsupported recommendation.

When sufficient data is available, the system creates a controlled prompt containing the relevant calculated information. The prompt instructs the AI model to interpret only the data supplied by LUNTI and prevents it from introducing unsupported external claims.

The resulting AI response is then processed and sanitized before being displayed to the user. In particular, numerical claims are checked against the underlying calculated summary so that displayed figures remain traceable to the user's recorded data. The finalized insight may be stored for future reference and is returned to the mobile application.

AI-generated insights are presented as recommendations or interpretations rather than guaranteed outcomes. Core financial calculations, including spending totals and budget status, remain deterministic and are performed by the application's backend rather than by the AI service.

6. HUMAN INTERFACE DESIGN

6.1 Overview of User Interface

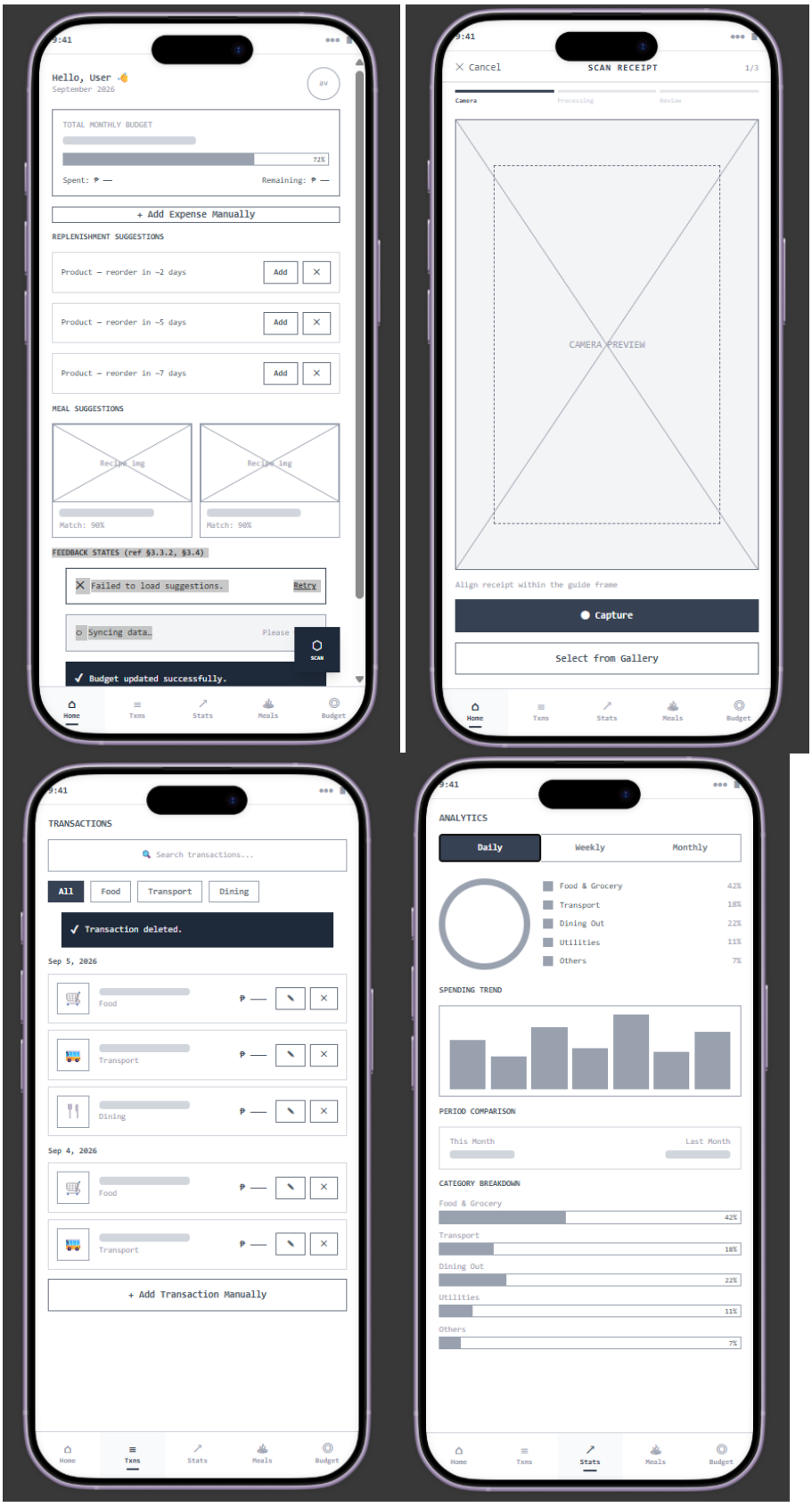
Lunti's interface is optimized for one-handed mobile use. Primary navigation is a bottom tab

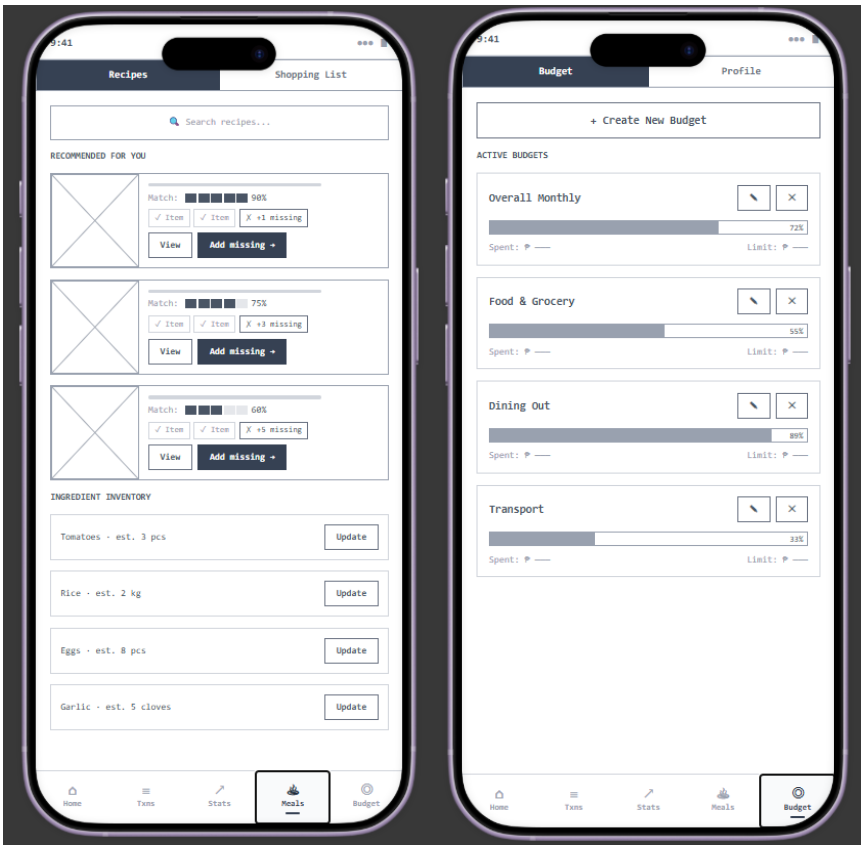
bar with five destinations: Home, Transactions, Analytics, Meals, and Budget/Profile. A prominent, always-reachable Scan Receipt action (floating action button) is available from Home, since receipt scanning is a primary method of recording purchases (SRS 3.7.1).

The system gives clear, immediate feedback for every operation: success confirmations after saving, visible loading/processing states during OCR and AI calls (which may take longer than typical requests), and understandable error messages with a retry option when a network or external-service call fails (SRS 3.3.2, 3.4).

6.2 Screen Images

https://www.figma.com/make/CmgwUKqiTo2LGBpsQomvWF/Create-low-fidelity-wireframe-sketch?t=nGsTGu89Z0Qzy9bt-1


Hello, User
TOTAL MONTHLY BUDGET
+ Add Expense Manually
REPLENISHMENT SUGGESTIONS
MEAL SUGGESTIONS
FEEDBACK STATES (ref §3.3.2, §3.4)
Failed to load suggestions.
Retry
Syncing data...
Budget updated successfully.
Cancel
SCAN RECEIPT
CAMERA PREVIEW
Capture
Select from Gallery
TRANSACTIONS
Transaction deleted.
+ Add Transaction Manually
ANALYTICS
Daily
Weekly
Monthly
SPENDING TREND
PERIOD COMPARISON
CATEGORY BREAKDOWN



6.3 Screen Objects and Actions

Screen	Key Objects	User Actions
Home	Budget summary card, Scan Receipt FAB, suggestion carousel	View summary; tap Scan Receipt; tap a suggestion to add/dismiss/modify
Scan Receipt	Camera preview, capture button, draft transaction form	Capture/select image; edit extracted fields; confirm or retry
Transactions	Transaction list item, filter/search bar	Search; filter by category/date; edit; delete; add manually
Analytics	Category chart, trend chart, comparison panel	Switch period (daily/weekly/monthly); drill into a category
Budget	Budget card with progress bar, Add Budget form	Create/edit/delete budget; set period and category
Meals	Recipe card, ingredient chip list	View recipe detail; add missing ingredients to shopping list
Shopping List	List item row, suggestion chip	Add/remove item; mark purchased; accept/dismiss a suggestion

7. REQUIREMENTS MATRIX

The table below traces each major SRS functional requirement group to the subsystem(s) and data entities responsible for satisfying it.

SRS §	Requirement Area	Responsible Subsystem(s)	Key Data Entities
3.1.1	Expense Management	Expense & Budget Service	Transaction, Category
3.1.2	Receipt Scanning and OCR	Receipt/OCR Orchestration Service	Receipt, TransactionLineItem
3.1.3	Budget Management	Expense & Budget Service	Budget, Transaction
3.1.4	Spending Analytics	Analytics & Pattern Service	Transaction, Category
3.1.5	Purchase Pattern Tracking	Analytics & Pattern Service	Product, PurchasePattern
3.1.6	Smart Replenishment	Replenishment & Shopping List Service	PurchasePattern, ReplenishmentSuggestion
3.1.7	Smart Shopping List	Replenishment & Shopping List Service	ShoppingList, ShoppingListItem
3.1.8	Ingredient Management	Ingredient & Recipe Service	IngredientInventory, Product
3.1.9	Recipe Recommendations	Ingredient & Recipe Service	Recipe, RecipeIngredient
3.1.10	AI-Powered Insights	AI Insights Service	Insight
3.1.11	User Feedback and Personalization	All services (feedback consumers)	Feedback
3.5	Security (Auth, RBAC, password, privacy)	API Layer + Supabase Auth	User
3.7.5	Notification Interface	Notification Service	Notification